Entornos de Desarrollo

Ejercicios Tema 6 – Documentación

C/ Literat Azorín, 1 46702 – GANDIA
Tfno. 962829430 – Fax. 962829431
Codi: 46004221 – CIF: Q-9655627-I
46004221@edu.gva.es
http://iesmariaenriquez.com

1º DAM

HUGO NAVARRO MIOTA

Entornos de Desarrollo	CFGS Desarrollo de Aplicaciones Multiplataforma	GENERALITAT VALENCIANA Conselleria d'Educació, Investigació, Cultura i Esport · Unió Europea Fons social europeu L'FSE inverteix en el teu futur · IES MARÍA ENRÍQUEZ	C/ Literat Azorín, 1 46702 – GANDIA Tfno. 962829430 – Fax. 962829431 Codi: 46004221 – CIF: Q-9655627-I 46004221@edu.gva.es http://iesmariaenriquez.com
Actividad	**Ejercicios Tema 6 – Documentación**		

Nombre:	Hugo Navarro Miota	Fecha:	27 / 4 / 2026	Nota:	

ÍNDICE

1 Ejercicio 1. Detección de errores en JavaDoc..... 3
2 Ejercicio 2. Documentación de una clase..... 5
3 Ejercicio 3. Selección de etiquetas adecuadas..... 6
4 Ejercicio 4. Evaluación de la documentación de un proyecto real..... 7
5 Ejercicio 5. Crear y documentar una aplicación..... 10
6 Ejercicio 6. Uso crítico de la IA para la documentación de código..... 15

1 Ejercicio 1. Detección de errores en JavaDoc

Lee el siguiente código e identifica **al menos 8 errores** relacionados con la documentación JavaDoc. Después, reescribe correctamente los comentarios.

ORIGINAL:

```java
/**
 * Clase
 * @author Germán Gascón <ggascon@gmail.com>
 */
public class Calculadora {

    /**
     * suma
     * @param x primer numero
     * @return
     */
    public int sumar(int a, int b) {
        return a + b;
    }

    /**
     * divide dos numeros
     * @param a
     * @param b divisor
     * @throws Exception
     */
    public int dividir(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("División por cero");
        }
        return a / b;
    }
}
```

CORREGIDO:

```java
/**
 * Clase que representa una calculadora básica
 *
 * @author Germán
 * @version 1.0
 */
public class Calculadora {

    /**
     * Realiza la suma de dos números.
     *
     * @param a Número que se va a sumar
     * @param b Número por el que se suma
     * @return Resultado de la suma
     */
    public int sumar(int a, int b) {
        return a + b;
    }

    /**
     * Realiza la división entera de dos números.
     *
     * @param a Número que se va a dividir
     * @param b Número por el que se divide
     * @return Resultado de la división
     * @throws ArithmeticException Si el divisor es cero
     */
    public int dividir(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("División por cero");
        }
        return a / b;
    }
}
```

2 Ejercicio 2. Documentación de una clase

Documenta completamente la siguiente clase utilizando JavaDoc.

```java
public class Temporizador {

    private int segundos;

    public Temporizador(int segundosIniciales) {
        this.segundos = segundosIniciales;
    }

    public void incrementar(int segundosExtra) {
        if (segundosExtra < 0) {
            throw new IllegalArgumentException("No se permiten valores
            negativos");
        }
        segundos += segundosExtra;
    }

    public void reiniciar() {
        segundos = 0;
    }

    public int getSegundos() {
        return segundos;
    }
}
```

3 Ejercicio 3. Selección de etiquetas adecuadas

Indica qué etiquetas JavaDoc deberían utilizarse para cada uno de los siguientes casos. No es necesario escribir toda la documentación completa; basta con indicar las etiquetas adecuadas y justificar brevemente la respuesta.

a) Método que recibe dos parámetros y devuelve un valor.

→ *@param* describiría cada parámetro (uno por línea) y *@return* devolvería un valor devuelto.

b) Método que puede lanzar `IllegalArgumentException`.

→ *@throws* indicaría excepciones que puede lanzar un método.

c) Clase pública de una biblioteca que empezó a existir en la versión 2.0.

→ *@since* indicaría desde qué versión existiría la clase pública de una biblioteca.

d) Método obsoleto que debe sustituirse por otro.

→ *@deprecated* indicaría que un elemento (en este caso, un método) estaría obsoleto.

e) Método cuya documentación necesita enlazar con otro método relacionado.

→ *@link* permitiría enlazar documentación relacionado de un método a otro.

f) Clase cuyo autor y versión deben mostrarse en la documentación.

→ *@author* indicaría el autor y *@version* mostraría la versión.

4 Ejercicio 4. Evaluación de la documentación de un proyecto real

Analiza la documentación del proyecto **jsoup**, una biblioteca Java muy conocida para trabajar con HTML y XML. El repositorio oficial en GitHub dispone de un README descriptivo, y el proyecto ofrece además una documentación API/Javadoc pública.

MATERIAL A CONSULTAR

- Repositorio: jhy/jsoup en GitHub.
- Documentación API/Javadoc oficial de jsoup.

ENUNCIADO

Evalúa la calidad de la documentación del proyecto respondiendo a las siguientes cuestiones:

1. ¿El README explica claramente qué es el proyecto y para qué sirve?

Exacto, es uno de los puntos fuertes de la documentación y explica que jsoup es una librería Java para trabajar con HTML / XML, está pensada para parsear documentos reales y es útil para scraping, limpieza y modificación de HTML. Su descripción se resume perfectamente en parsear HTML desde URL, archivo o string, extraer información con selectores CSS, modificar nodos y atributos, limpiar HTML inseguro como XSS y generar HTML limpio. Por tanto, cualquier desarrollador entiende rápidamente qué hace, para qué sirve y cuándo usarla.

2. ¿Incluye instrucciones de uso o ejemplos?

Sí, se incluyen ejemplos bastante bien organizados, ya que por ejemplo, incluyen un ejemplo típico de uso para conectarse a una URL, parsear el documento y seleccionar elementos. Además, dispone de un Cookbook con ejemplos prácticos para cargar documentos, usar selectores CSS, consultar XPath, modificar elementos, limpiar HTML y trabajar con sesiones HTTP.

3. ¿La documentación API es fácil de navegar?

En general sí. Su documentación está estructurada e incluyen Overview, paquetes, clases, interfaces, índice alfabético, buscador, árbol de herencia, páginas de uso y clases relacionadas. Tiene un punto mejorable que sería la interfaz clásica de JavaDoc, ya que es muy funcional y es menos amigable visualmente sobre la documentación moderna tipo Swagger, MDN o Spring Docs.

4. ¿Los nombres de clases y métodos resultan claros?

Los nombres de los métodos resultan claros y semánticos como por ejemplo: Jsoup.parse(), connect(), select(), text(), html(), attr() y clean() que describen exactamente la acción que realizan. También las clases son intuitivas y algunos ejemplos a describir son: Document, Element, Elements, Connection y Safelist que facilitan su uso incluso sin consultar la documentación constantemente.

5. ¿La documentación está orientada a desarrolladores?

Completamente está escrita para desarrolladores Java. Se nota en varios aspectos como: ejemplos de código, firmas completas de métodos, tipos de retorno, excepciones, parámetros documentados y enlaces entre clases relacionadas.

6. ¿Qué aspectos mejorarías del README?

Agregaría una guía rápida de instalación más visible para incluir ejemplos directos con Maven y Gradle; también agregaría secciones de casos de uso reales como scraping web, limpieza de formularios, extracción SEO y procesamiento de emails HTML; también agregaría una tabla con resúmenes sobre las funcionalidades de funciones a métodos principales para ver el README más rápido de consultar; y por último agregaría unos cuantos ejemplos más de XML, ya que se enfoca más en HTML que en XML y estaría bien equilibrarlo un poco.

7. ¿Qué aspectos mejorarías del JavaDoc o documentación API?

Agregaría más ejemplos por método, porque hay métodos que se describen como pueden, pero no siempre muestran ejemplos disponibles, con lo cual ayudaría bastante unos cuantos ejemplos; también agregaría una mejora a la experiencia visual, porque se deberían centrar en modernizar un poco el JavaDoc o documentación API para que no fuese muy densa; también agregaría más documentación conceptual, porque la API documenta muy bien qué hace cada método, pero faltaría el por qué usarlo para reforzar la documentación API; y por último agregaría más tutoriales enlazados desde la API, porque sería ideal que desde clases clave como Jsoup o Element hubiesen enlaces directos al Cookbook para mejorar la navegación entre referencia y aprendizaje.

5 Ejercicio 5. Crear y documentar una aplicación

Implementa en Java una aplicación que simule partidos de fútbol simplificados por consola.
La aplicación deberá contar con un objeto **Narrador** encargado de describir cada jugada.

ESTRUCTURA

JUGADOR

Cada jugador tendrá:

- dorsal
- nombre y apellidos
- portero (0-100)
- defensa (0-100)
- regate (0-100)
- pase (0-100)
- tiro (0-100)

EQUIPO

Cada equipo tendrá:

- nombre
- array de 11 jugadores
- cantidad de goles marcados en el partido

PELOTA

La pelota tendrá:

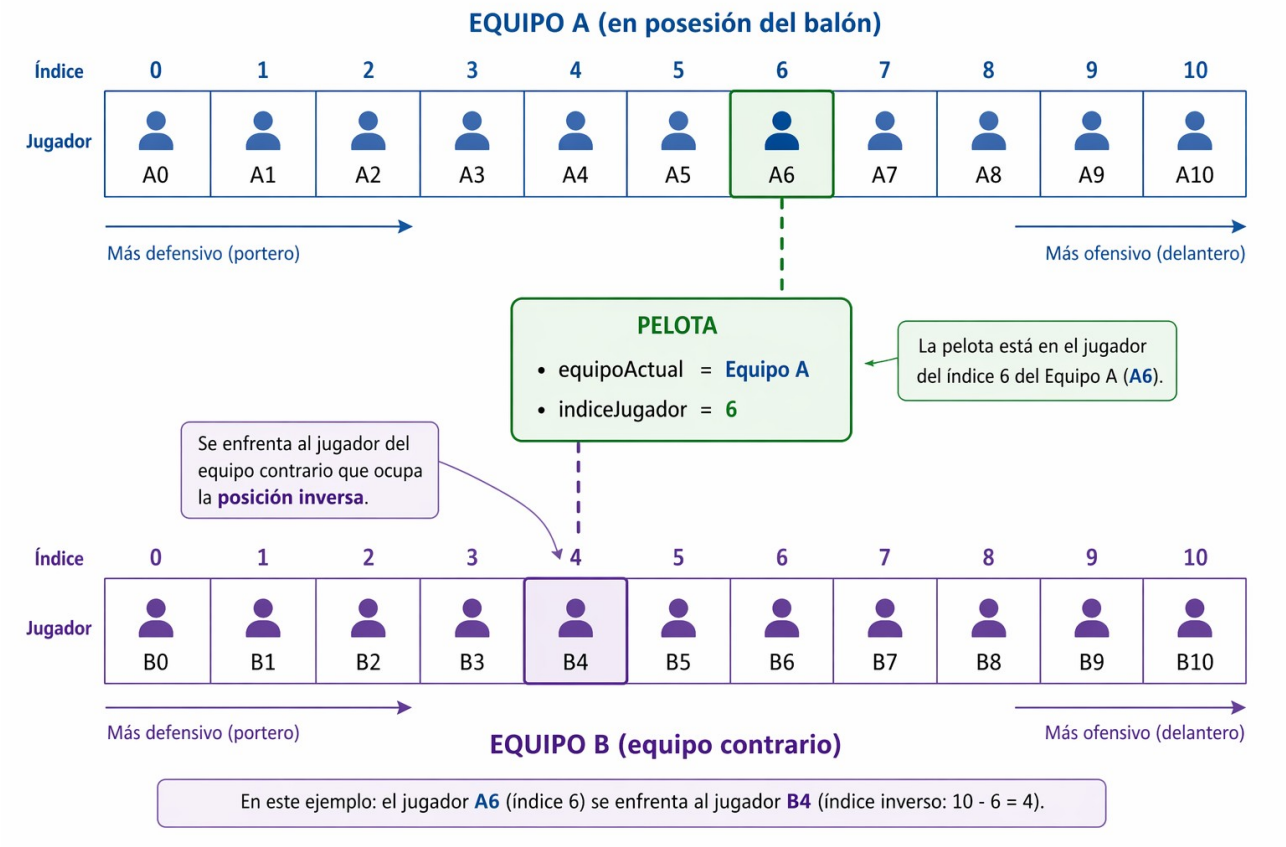
- referencia al equipo en posesión
- índice del jugador que la posee (0...10)

DESARROLLO DEL JUEGO

El objetivo es que la pelota vaya avanzando posiciones en el array hasta llegar a una posición de tiro para meter gol.
El jugador con la pelota se enfrenta al rival en posición inversa:

- índice 10 con índice 0
- 9 con 1
- 8 con 2
- etc.

Excepto en el caso del tiro, donde el jugador rival siempre será el portero (ver siguiente apartado).



ACCIONES POSIBLES

En cada turno, el jugador con la pelota elegirá aleatoriamente una acción válida que puede ser:

- **Pase**

  Se comparan las propiedades:

  - ***pase*** del atacante
  - ***defensa*** del rival

  Si tiene éxito, el balón pasa al siguiente compañero del array, es decir, la pelota avanza una posición en el array.

  Restricción: el jugador con índice 10 (último), no puede realizar pases, solo tiro.

- **Regate**

  Se compara:

  - ***regate*** del atacante
  - ***defensa*** del rival

  Si tiene éxito, el jugador avanza una posición (se intercambia con el compañero siguiente en el array) y la pelota avanza una posición, de esta forma continua apuntando al mismo jugador pero que ahora está una posición más adelantada.

  Restricción: el jugador con índice 10 (último), no puede realizar regates, solo tiro.

- **Tiro**

  Solo se pueden jugadores cuyo índice en el array sea ≥ 7.

  Se compara:

  - ***tiro*** ajustado por distancia/índice. Por cada posición alejada del índice 10, el tiro se reduce un 10%. El cálculo sería: tiroCalculado = tiroBase * (1 – (10 – índice) * 0.10)
  - ***portero*** rival (jugador 0)

ALEATORIEDAD

Todas las acciones se les aplicará una variación aleatoria entre **-10%** y **+10%** para que las jugadas no produzcan siempre el mismo resultado.

PARTIDO

- Se generan 2 equipos con 11 jugadores aleatorios cada uno usando la librería **DataFaker** para generar los nombres y apellidos.
- Se sortea el saque inicial para decidir qué equipo tiene la pelota inicialmente.
- El partido dura 3 minutos reales. Ayúdate de: System.currentTimeMillis() tomando valores al principio y después en cada acción, para calcular el tiempo total transcurrido.
- Entre acciones habrá pausa de 2 segundos para que dé tiempo a seguir el partido: Thread.*sleep*(2000);

NARRADOR

Debe narrar al menos:

- pases
- regates
- robos
- tiros
- paradas
- goles
- resultado final

Por ejemplo algunos mensajes del narrador podrían ser:

"Juan García combina con Asier González"
"Pedro Martínez intenta el pase pero Unai López le adivina la acción y recupera la pelota"
"Diego Moncho dispara y... Pablo Rivero muy atento atrapa la pelota sin problemas"

PASOS

1. Implementa las clases necesarias para definir la lógica y la funcionalidad requerida del simulador.
2. Documenta con JavaDoc: clases, atributos, constructores, métodos públicos y excepciones.
3. Genera el JavaDoc final, para ello debes crear una tarea Gradle para configurar la generación de la documentación.

6 Ejercicio 6. Uso crítico de la IA para la documentación de código

Toma una clase Java creada por ti que tenga al menos 4 atributos y 4 métodos, y utiliza una herramienta de IA para generar una primera versión de la documentación. Después, realiza una revisión crítica siguiendo los siguientes pasos (realiza una captura de pantalla de cada paso):

1. Copia el código en una herramienta de IA y solicita una propuesta de documentación JavaDoc.

Java

```java
/**
 * La clase ComprobacionIA permite realizar operaciones matemáticas básicas
 * (suma, resta, multiplicación y división) entre dos números.
 * También almacena el resultado de la última operación realizada
 * junto con el tipo de operación.
 *
 * @author
 * @version 1.0
 */
public class ComprobacionIA {

    /** Primer número de la operación */
    private double numero1;

    /** Segundo número de la operación */
    private double numero2;

    /** Resultado de la última operación realizada */
    private double resultado;

    /** Nombre de la última operación realizada */
    private String operacion;
```

Java

```java
/**
 * Constructor de la clase ComprobacionIA.
 *
 * @param numero1 Primer número
 * @param numero2 Segundo número
 */
public ComprobacionIA(double numero1, double numero2) {
    this.numero1 = numero1;
    this.numero2 = numero2;
}

/**
 * Realiza la suma de los dos números.
 *
 * @return Resultado de la suma
 */
public double suma() {
    resultado = numero1 + numero2;
    operacion = "Suma";
    return resultado;
}
```

Java

```java
/**
 * Realiza la resta de los dos números.
 *
 * @return Resultado de la resta
 */
public double resta() {
    resultado = numero1 - numero2;
    operacion = "Resta";
    return resultado;
}

/**
 * Realiza la multiplicación de los dos números.
 *
 * @return Resultado de la multiplicación
 */
public double multiplicacion() {
    resultado = numero1 * numero2;
    operacion = "Multiplicación";
    return resultado;
}
```

Java

```java
    /**
     * Realiza la división de los dos números.
     *
     * @return Resultado de la división
     * @throws ArithmeticException Si se intenta dividir entre cero
     */
    public double division() {
        if (numero2 != 0) {
            resultado = numero1 / numero2;
            operacion = "División";
            return resultado;
        } else {
            throw new ArithmeticException("ERROR: División entre cero.");
        }
    }

    /**
     * Muestra por consola la última operación realizada
     * junto con su resultado.
     */
    public void mostrarResultado() {
        System.out.println("Operación: " + operacion);
        System.out.println("Resultado: " + resultado);
    }
}
```

2. Comparar el resultado con el código real y detectar errores o problemas como:
   - descripciones imprecisas,
   - parámetros mal interpretados,
   - excepciones inventadas o ausentes,
   - documentación redundante,
   - falta de contexto,
   - uso incorrecto de etiquetas.

3. Corregir manualmente la documentación

PREGUNTAS DE REFLEXIÓN

¿La IA entendió correctamente el propósito del código?

Entendió bastante bien el propósito del código y de la documentación.

¿La documentación generada era demasiado genérica?

Para serte sincero, fue demasiado genérica.

¿Había información incorrecta o inventada?

No había información incorrecta o inventada, todo parece estar correcto.

¿Qué partes has tenido que corregir?

En principio no se tendría que corregir mucho porque es una documentación con JavaDoc bastante legible. Lo único máximo que se podría haber corregido es la parte del "@author" para agregar el Autor correspondiente, pero por lo demás todo parece correcto y legible.

¿En qué situaciones puede ayudar la IA y en cuáles no conviene confiar sin revisar?

Puede ayudar la IA si sabe los conceptos del código que le pasas o que le pidas. No conviene confiar si no realizas una revisión adecuada, porque podría contener errores o algún fallo leve que a lo mejor no nos damos cuenta y se podría liar cierta parte.